

OOP Principles

Practical Java Examples & Student

Tasks

Abstraction

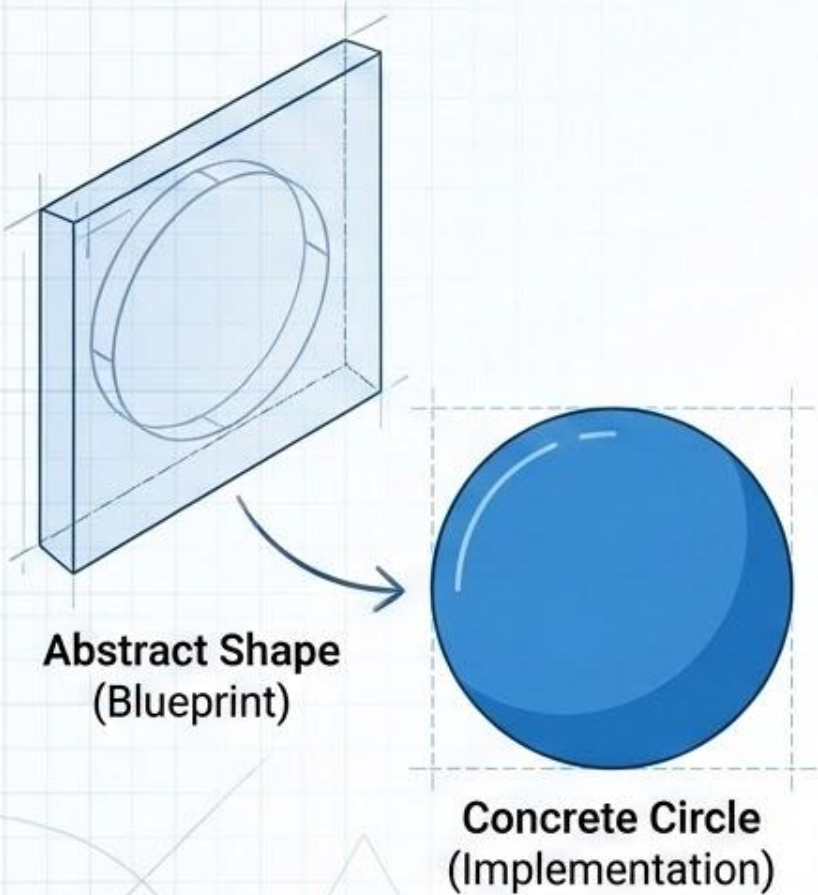
Inheritance

Polymorphism

Abstraction in Java

Example: Shapes

We define a blueprint for Shape but don't implement the draw() logic yet. Each specific shape must define how it draws itself.



```
abstract class Shape {  
    abstract void draw();  
}  
  
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing Circle");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Shape s = new Circle();  
        s.draw(); // Output: Drawing Circle  
    }  
}
```



Key Concept

You cannot create an instance of Shape directly (new Shape() is illegal).

You must instantiate concrete classes like Circle.

Student Task: Bank System

Assignment

- Create an abstract class BankAccount.
- Add an abstract method calculateInterest().
- Create two subclasses: SavingsAccount (interest 5%) and CurrentAccount (interest 0%).
- In a main method, create objects for both and print their calculated interest.

Tip: Think about what properties all accounts share (e.g., balance, accountNumber) and put them in the abstract class.



Inheritance in Java

```
class Employee {
    String name;
    Employee(String n) {
        this.name = n;
    }
    void work() {
        System.out.println(name + " is working");
    }
}

class Manager extends Employee {
    int teamSize;
    Manager(String n, int size) {
        super(n); // Call parent constructor
        this.teamSize = size;
    }
    @Override
    void work() {
        System.out.println(name + " manages " + teamSize + " people");
    }
}
```



Extends & Super



Employee
(Parent)



Manager
(Child)

The Manager class reuses the name field from Employee but adds its own teamSize.



Watch Out

If the parent class has a constructor with parameters, the child class must call it using `super()` as the first line in its own constructor.

Student Task: Device Hierarchy

Assignment

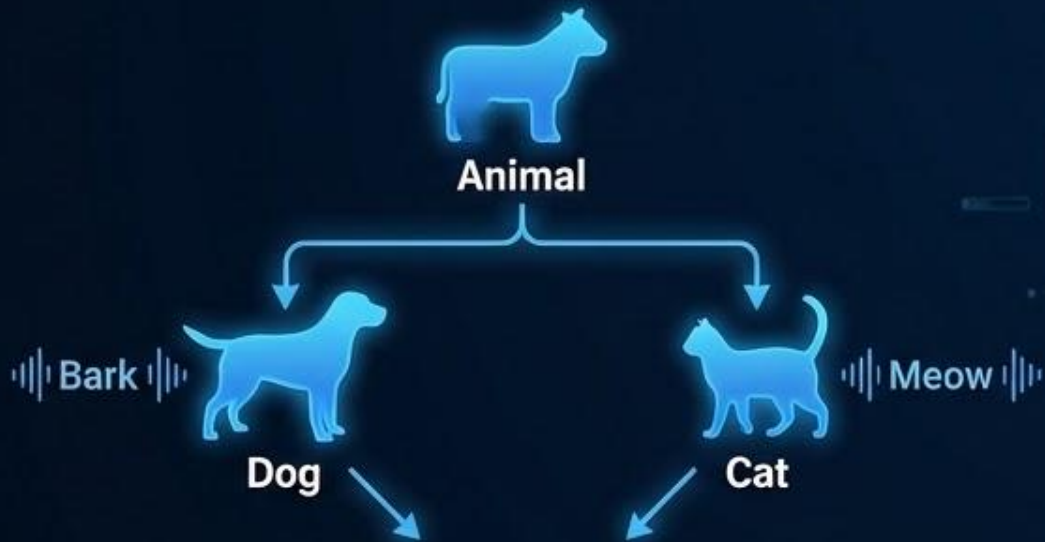
- Create a base class `ElectronicDevice` with properties `brand` and method `powerOn()`.
- Create a subclass `SmartPhone` that adds `batteryLevel` and overrides `powerOn()` to print "Booting Android/iOS...".
- Create another subclass `Laptop` with `ramSize`.

Challenge: Add a constructor to the base class and ensure subclasses call it correctly.



Polymorphism in Java

↻ Runtime Polymorphism



Dynamic Method Dispatch

We treat different objects (Cat, Dog) as a generic Animal. The specific behavior runs based on the actual object type at runtime. This allows for flexible code where you can add new animal types without changing the main logic.

```
class Animal {
    void sound() { System.out.print("..."); }
}

class Dog extends Animal {
    @Override
    void sound() { System.out.print("Bark"); }
}

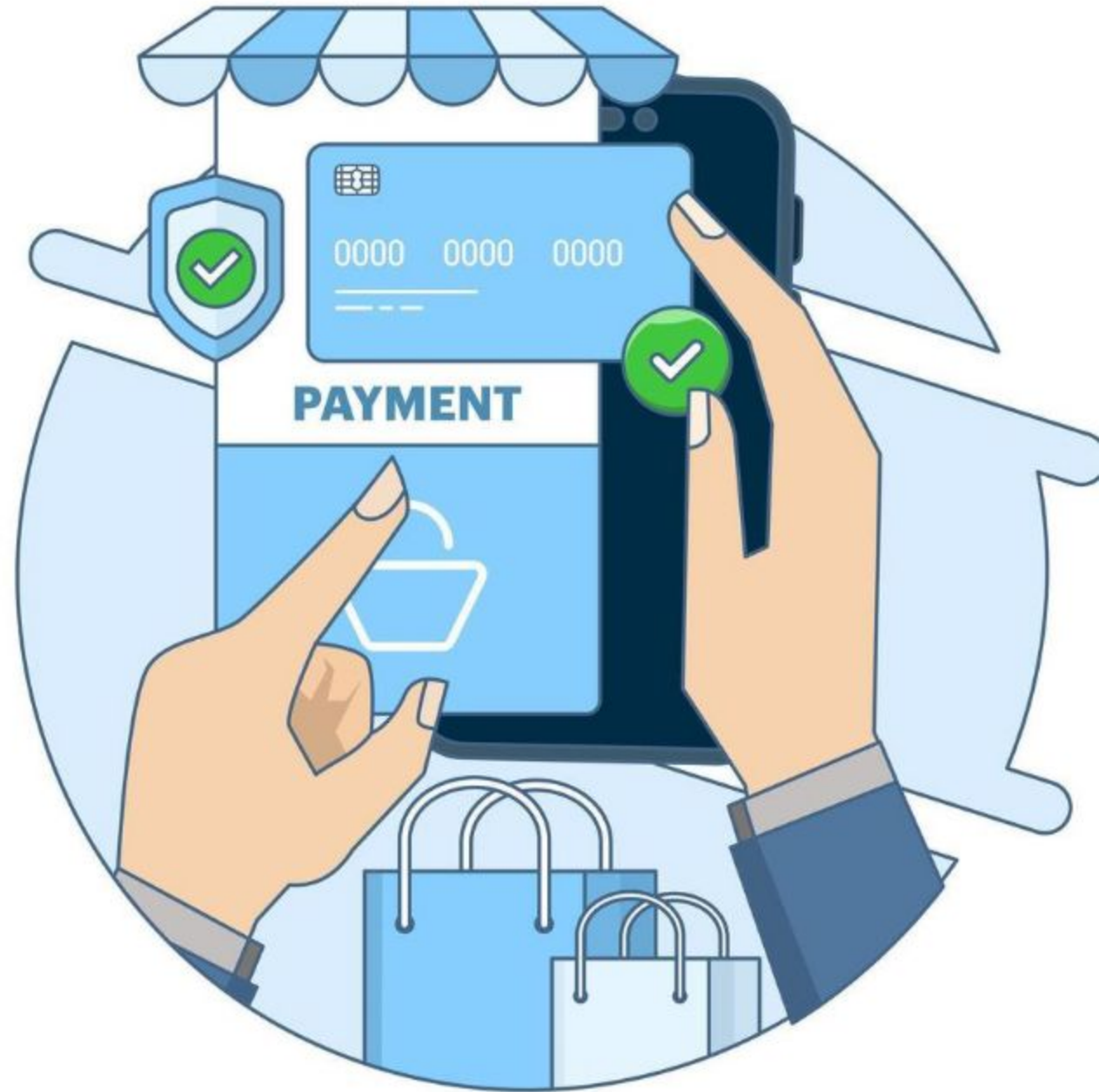
class Cat extends Animal {
    @Override
    void sound() { System.out.print("Meow"); }
}

public class Zoo {
    public static void main(String[] args) {
        Animal myPet = new Dog(); // Upcasting
        myPet.sound(); // Output: Bark
        myPet = new Cat();
        myPet.sound(); // Output: Meow
    }
}
```

Student Task: Payment Gateway

Assignment

- Define an interface (or abstract class) `PaymentMethod` with a function `pay(double amount)`.
- Implement `CreditCard` and `PayPal` classes that print different messages (e.g., "Swiping card..." vs "Redirecting to PayPal...").
- Create a `ShoppingCart` class that takes a `PaymentMethod` and processes a payment.



Association & Composition

```
class Engine {  
    void start() {  
        System.out.println("Engine starting...");  
    }  
}  
  
class Car {  
    private final Engine engine; // Composition  
  
    Car() {  
        this.engine = new Engine(); // Car owns Engine  
    }  
  
    void drive() {  
        engine.start();  
        System.out.println("Car moving");  
    }  
}
```

Composition (Strong)

In this example, the Car creates the Engine. If the Car is destroyed, the Engine is also destroyed (conceptually). This is a "Part-of" relationship.

Aggregation (Weak)

If the Engine was passed into the constructor (new Car(myEngine)), it would be Aggregation, as the engine could belong to another car later.

Student Task: University System

Assignment

- **Composition:** Create a University class that contains a list of Department objects. If the University closes, Departments cease to exist.
- **Aggregation:** Create a Professor class. Professors belong to a Department but can exist independently (e.g., they can move to another university).



Delegation in Java

Passing the Buck



Instead of inheriting behavior, a class keeps an instance of another class and calls its methods to do the work. Here, the TicketBooking class doesn't know how to process payments itself; it delegates that task to a PaymentProcessor.

```
class PaymentProcessor {  
  
    void process(double amt) {  
        System.out.println("Paid: $" + amt);  
    }  
}  
  
class TicketBooking {  
    PaymentProcessor processor =  
        new PaymentProcessor();  
  
    void bookTicket(double price) {  
        // Delegation happening here  
        processor.process(price);  
        System.out.println("Ticket Confirmed");  
    }  
}
```


Student Task: Event Planner

Assignment

- Create a Chef class with a method `cookFood()`
- Create a Musician class with a method `playMusic()`.
- Create an EventPlanner class. It should NOT implement cooking or music logic itself.
- Instead, the EventPlanner should have instances of Chef and Musician and **delegate** the work when `hostParty()` is called.

